

Project 1: Robot Code Documentation

Team members: *[names]*

[date]

1 Overview

- *[One paragraph: what the code does.]* Two ROS 2 Python nodes in package `proj1`: `reactive_node.py` (class `SubsumptionController`) drives the TurtleBot 4; `occupancy_mapper.py` (class `OccupancyMapper`) builds the map. They run independently and share only sensor topics.
- *[Figure 1: node/topic graph]* – `/scan`, `/odom`, `/hazard_detection`, `/teleop_cmd` → controller → `/cmd_vel`; `/scan`, `/odom`, TF → mapper → `/map`.

2 Reactive Architecture

2.1 Which architecture

- *[Name it.]* Priority-based pseudo-subsumption in the style of Jones et al.: every behavior proposes a command or abstains, and the highest-priority proposal wins.
- *[Contrast briefly with Brooks' subsumption (suppression and inhibition wires between layers) and with schema theory (vector summation of motor schemas), and say what we did not do.]*

2.2 Why we chose it

- *[Reasons, e.g.: the brief gives a strict priority ordering, which maps directly onto a fixed arbitration list; behaviors are independent and easy to test one at a time; no vector tuning as in schemas; simple to explain and debug.]*

2.3 Where the code embodies it

- `self.behaviors` tuple: the priority order, highest first.
- `control_loop()`: the arbiter; publishes the first non-`None` command each cycle (10 Hz).
- Each behavior is one method returning a command or `None`.
- *[Figure 2: priority stack diagram, halt on top, forward at the bottom, arbiter on the right.]*

3 Behaviors

[One short paragraph per behavior, or rely mostly on the table. Point out reflex vs. fixed action pattern explicitly.]

#	Method	Trigger	Response / type
1	halt	any bump in <code>/hazard_detection</code>	stop; reflex
2	teleop	key command < 0.5 s old	pass through
3	escape	both front arcs < 1 ft, nearly equal	turn $180^\circ \pm 30^\circ$; FAP
4	avoid	either front arc < 1 ft	turn away from closer side; reflex
5	random_turn	1 ft of forward travel	turn $U(-15^\circ, 15^\circ)$
6	forward	always	drive at 0.2 m/s

Table 1: Behaviors, highest priority first.

- *[Explain why escape is a fixed action pattern: the target heading is latched in `turn_target`/`turn_owner` and the turn finishes even after the obstacle disappears.]*
- *[Explain why avoid is a reflex: it is recomputed every cycle and stops as soon as nothing is within 1 ft.]*
- *[Note that avoid cancels a pending random turn.]*

4 Data Structures

4.1 Controller

- Sensor state: `bumper_hit`, `front_left_dist`, `front_right_dist`, cached `scan_yaw_offset`.
- Odometry state: `current_yaw`, last position, `forward_travel` accumulator.
- Teleop state: last twist and its arrival time.
- Latched turn: `turn_target` (absolute heading) and `turn_owner` (which behavior owns it).
- ROS parameters for every threshold (speeds, 1 ft distance, $\pm 30^\circ$ front arc, symmetry tolerance, turn angles). *[Say why: the TA can retune without editing code.]*

4.2 Mapper

- 2-D NumPy `float32` array of log-odds, 0.05 m cells, 20 m $\times$ 20 m centered on the start pose.
- Boolean `seen` mask so never-observed cells stay unknown (-1).
- Cached lidar mount pose (x, y, yaw) in the base frame.

5 Algorithms

- **Front-obstacle detection:** convert each beam angle into the robot frame using the lidar's TF yaw offset, keep the minimum range in the left (0 to $+30^\circ$) and right (-30° to 0) arcs. *[Why TF: robust if the TA moves or rotates the lidar.]*
- **Symmetry test:** both arcs < 1 ft and $|d_L - d_R| < \text{tolerance} \Rightarrow \text{escape}$; otherwise avoid.
- **Distance for random turn:** project each odometry step onto the heading; only positive (forward) motion counts.

- **Latched turns:** target = current yaw + angle, wrapped to $[-\pi, \pi]$; rotate at fixed speed until within 3° .
- **Occupancy mapping:** per scan, transform the lidar pose into the odom frame, ray-cast each beam at half-cell steps, add L_{free} to traversed cells and L_{occ} to the end cell, clamp to ± 5 , publish $p = 1 - 1/(1 + e^l)$ scaled to 0–100. *[Cite the log-odds occupancy-grid formulation.]*

6 Design Decisions and Limitations

- *[Map is in the odom frame, so it drifts with odometry; no localization, as the brief does not require navigation.]*
- *["1 ft in front" is measured from the lidar.]*
- *[Halt outranks teleop, so a pressed bumper blocks keyboard control too.]*
- *[Create 3 built-in bump reflexes are disabled at launch so that halt is in charge.]*
- *[Coding standards: ROS 2 Python style, flake8 and pep257 tests, object-oriented nodes.]*

7 Sources and Credit

[Required by Section 6 of the assignment. List every source of code or text and which parts are original, changed, or taken from where (including tools used to help write code or documents). List who on the team wrote which parts.]